Java(sem)-1

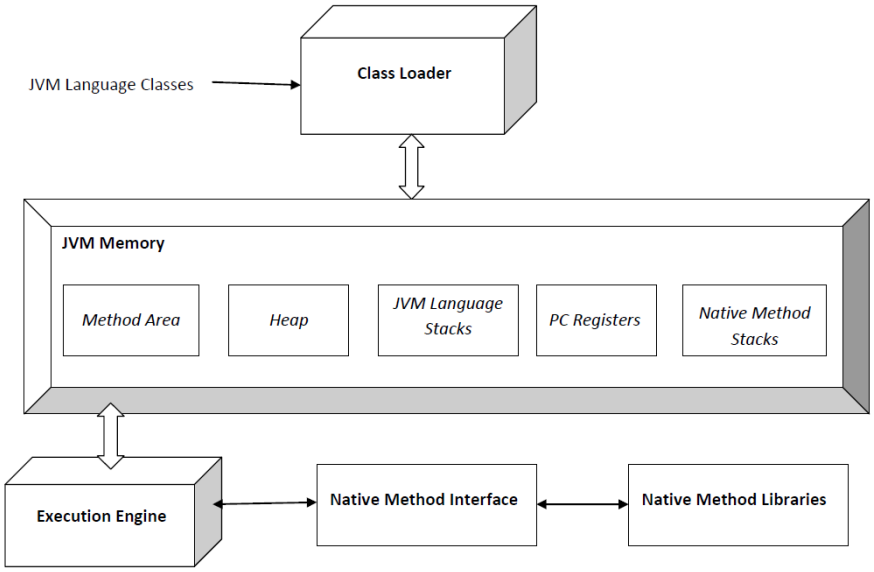
1. Describe the JVM architecture with a neat diagram. Explain the role of Class Loader, Memory Areas, and Execution Engine?

JVM Architecture

Definition

The **Java Virtual Machine (JVM)** is a part of the Java Runtime Environment that executes Java bytecode and enables platform independence ("Write Once, Run Anywhere").

The Java Virtual Machine (JVM) is a core component of the Java Runtime Environment (JRE) that allows Java programs to run on any platform without modification. JVM acts as an interpreter between Java bytecode and the underlying hardware, providing Java's famous Write Once, Run Anywhere (WORA) capability.



Architecture Overview

The JVM consists of three major parts:

- Class Loader Subsystem
- Runtime Memory Areas
- Execution Engine

It also interacts with JNI and native libraries.

Class Loader Subsystem

Role

The Class Loader loads .class files into memory and prepares them for execution.

Working

- **Loading**: Reads bytecode and creates class objects in memory
- **Linking**:
  - Verification → checks correctness of bytecode
  - Preparation → allocates memory for static variables
  - Resolution → converts symbolic references into actual references
- **Initialization**:
  - Assigns values to static variables
  - Executes static blocks

Types

- Bootstrap Class Loader
- Extension Class Loader
- Application Class Loader

JVM Memory Areas (Runtime Data Areas)

Method Area

Stores class-level data such as metadata, methods, and static variables. Shared across all threads.

Heap

Stores objects and instance variables. It is the main memory used for dynamic allocation and garbage collection.

Java Stack

Each thread has its own stack. It stores method calls and local variables using stack frames.

PC Register

Stores the address of the currently executing instruction for each thread.

Native Method Stack

Used for executing native methods written in languages like C or C++.

Execution Engine

Role

Executes the bytecode loaded into memory.

Components

- **Interpreter**: Executes bytecode line by line (slower)
- **JIT Compiler**: Converts bytecode into native code for faster execution
- **Garbage Collector**: Removes unused objects from heap memory

---

Flow of Execution

- Java source code is compiled into bytecode
- Class Loader loads the class
- Memory areas store required data
- Execution Engine runs the program

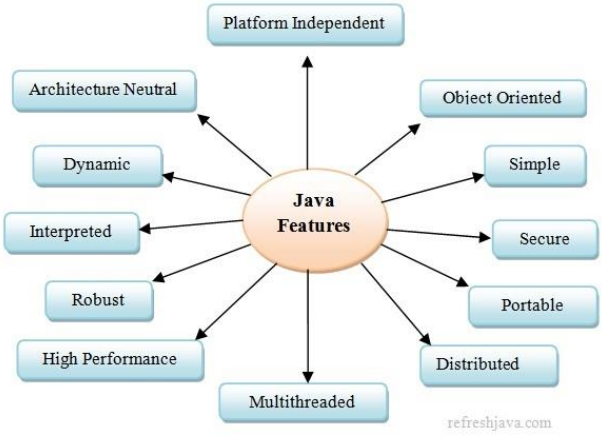
Key Points

- JVM ensures platform independence
- Provides automatic memory management
- Ensures security through bytecode verification
- Heap and Method Area are shared; Stack is thread-specific

Q. Discuss the features of Java (Java Buzzwords) in detail. How do these features make Java robust and secure?

Definition

Java Buzzwords are the key features of Java that define its design goals such as simplicity, security, robustness, and platform independence.



Features of Java

Simple

- Easy to learn and use
- Removes complex features like pointers and operator overloading
- Syntax similar to C/C++ but cleaner

Object-Oriented

- Based on classes and objects
- Supports encapsulation, inheritance, polymorphism
- Promotes code reusability and modularity

Platform Independent

- Java code is compiled into bytecode
- Runs on any system with JVM ("Write Once, Run Anywhere")

Secure

- No direct memory access (no pointers)
- Bytecode verification prevents illegal operations
- Provides sandbox environment for execution

Robust

- Strong memory management
- Automatic Garbage Collection
- Exception handling mechanism
- Eliminates common errors like memory leaks

Multithreaded

- Supports multiple threads execution
- Improves performance and responsiveness

Portable

- Same bytecode runs on different platforms
- Fixed size of data types ensures consistency

High Performance

- Uses JIT compiler for faster execution

- Converts bytecode into native machine code

Distributed

- Supports networking APIs
- Used for distributed applications

Dynamic

- Classes can be loaded at runtime
- Supports dynamic memory allocation

How Java is Robust

Java becomes robust due to:

- **Strong memory management** → Garbage Collector removes unused objects
- **Exception handling** → Handles runtime errors effectively
- **No pointers** → Avoids memory corruption
- **Type checking** → Errors detected at compile time
- **JVM checks bytecode** → Prevents invalid operations

How Java is Secure

Java ensures security through:

- **Bytecode verification** → Detects malicious code
- **No pointer manipulation** → Prevents unauthorized memory access
- **Security Manager & Class Loader** → Controls access to resources
- **Sandbox environment** → Restricts untrusted code execution
- **Runtime checks** → Ensures safe execution

Conclusion

Java's features like automatic memory management, strong type checking, and secure execution environment make it both **robust and secure**, suitable for large-scale and reliable applications.

Q. Explain Java source file structure and compilation process with an example

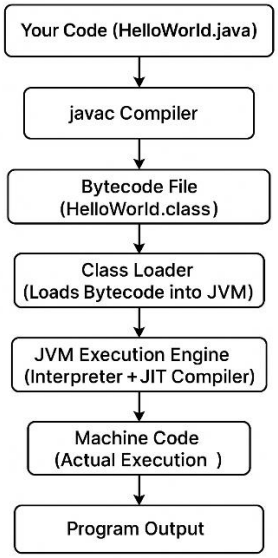
---

Answer

Java Source File Structure

A Java source file is saved with the **.java** extension. It follows a **fixed order**.

A Java source file (.java) is organized in a specific order. Not all parts are mandatory, but the general structure is:



Package Declaration

- Defines the package to which the class belongs
- Must be the first statement (if present)

package mypackage;

Import Statements

- Used to include predefined classes and packages

import java.util.*;

Interface Section (optional)

- Used to declare interfaces

Class Definition

- Contains variables, methods, constructors, and main method

- Only one public class is allowed per file
- File name must match the public class name

Basic Structure Example

```
package mypackage;


import java.util.*;


class Helper {
    void show() {
        System.out.println("Helper class method");
    }
}


public class Demo {
    public static void main(String[] args) {
        Helper h = new Helper();
        h.show();
        System.out.println("Main method executed");
    }
}
```

Compilation and Execution Process

Step 1: Writing Source Code

- Program is written in a .java file

Step 2: Compilation

- Java compiler (javac) converts source code into bytecode

```
javac Demo.java
```

- Output: Demo.class (bytecode)

Step 3: Class Loading

- JVM loads .class file using Class Loader

Step 4: Bytecode Verification

- JVM checks for security and correctness

Step 5: Execution

- JVM executes bytecode using interpreter or JIT compiler

java Demo

Example with Output

```
public class Hello {
   public static void main(String[] args) {
      System.out.println("Hello Java");
   }
}
```

Compilation

```
javac Hello.java
```

Execution

```
java Hello
```

Output

```
Hello Java
```

Key Points

- File name must match public class name
- Compilation generates bytecode (.class)
- JVM executes bytecode, not source code
- Java is platform independent due to bytecode

Conclusion

Java source file structure provides organized program design, and its compilation process ensures platform-independent execution through bytecode and JVM.

Q6. Discuss operators in Java. Explain different types of operators with suitable examples.

Answer

Definition

Operators in Java are special symbols used to perform operations on variables and values. They help in performing calculations, comparisons, and logical decision-making.

Types of Operators in Java

Arithmetic Operators

Used to perform mathematical operations such as addition, subtraction, multiplication, division, and modulus.
Operators: + - * / %

Example:

```
int a = 10, b = 3;
System.out.println(a + b); // 13
System.out.println(a % b); // 1
```

Relational Operators

Used to compare two values and return a boolean result (true or false).
Operators: == != > < >= <=

Example:

```
int x = 5, y = 10;
System.out.println(x < y); // true
```

Logical Operators

Used to combine boolean expressions.
Operators: && || !

Example:

```
boolean a = true, b = false;
System.out.println(a && b); // false
System.out.println(a || b); // true
System.out.println(!a);     // false
```

Assignment Operators

Used to assign and update values.
Operators: = += -= *= /= %=

Example:

```
int a = 10;
a += 5;
System.out.println(a); // 15
```

Unary Operators

Operate on a single operand.
Operators: + - ++ -- !

Operator	Meaning
+	Unary plus
-	Unary minus
++	Increment
--	Decrement
!	Logical NOT

Example:

```
int a = 5;
a++;
System.out.println(a); // 6
```

Bitwise Operators

Operate at the binary level.
Operators: & | ^ ~ << >> >>>

Operator	Meaning
&	Bitwise AND
^	Bitwise XOR
~	Bitwise Complement
<<	Left shift
>>	Right shift
|	Bitwise OR

Example:

```
int a = 5, b = 3;
System.out.println(a & b); // 1
```

Ternary Operator

A shorthand form of if-else.

Syntax:

```
condition ? value1 : value2;
```

Example:

```
int a = 10, b = 20;
int max = (a > b) ? a : b;
System.out.println(max); // 20
```

Bitwise Operators (Detailed Explanation)

Bitwise operators work on binary representation (bits) of integers (byte, short, int, long).
They are useful in system programming and performance optimization.

Let:

a = 5 → 0101
b = 3 → 0011

Bitwise AND (&)

Returns 1 only if both bits are 1.

```
0101
0011
-----
0001 → 1
```

Bitwise OR (|)

Returns 1 if at least one bit is 1.

```
0101
0011
-----
0111 → 7
```

Bitwise XOR (^)

Returns 1 if bits are different.

```
0101
0011
-----
0110 → 6
```

Bitwise Complement (~)

Inverts all bits.

Formula:

~N = -(N + 1)

Example:

~5 = -6

Shift Operators

Left Shift (<<)

Shifts bits to the left (adds zeros on right).
Equivalent to multiplication by 2^n.

Example:

5 << 2 = 20

Right Shift (>>)

Shifts bits to the right (preserves sign).
Equivalent to division by 2^n.

Example:

20 >> 2 = 5

Unsigned Right Shift (>>>)

Shifts bits to the right and fills with 0.

Example Program

```
public class BitwiseDemo {
   public static void main(String[] args) {
      int a = 5, b = 3;

      System.out.println("a & b = " + (a & b));
      System.out.println("a | b = " + (a | b));
      System.out.println("a ^ b = " + (a ^ b));
      System.out.println("~a = " + (~a));
      System.out.println("a << 1 = " + (a << 1));
      System.out.println("a >> 1 = " + (a >> 1));
   }
}
```

Output

a & b = 1
a | b = 7
a ^ b = 6
~a = -6
a << 1 = 10
a >> 1 = 2

Applications of Bitwise Operators

- Bit masking
- Cryptography
- Data compression
- Performance optimization

Conclusion

Bitwise operators allow manipulation of data at the binary level, making programs faster and efficient. They are important in low-level and performance-critical applications.

Q. Explain type conversion and type casting in Java. Differentiate between implicit and explicit conversion.

Type Conversion in Java (Definition)

Type conversion is the process of converting one data type into another data type. It is required when different data types are used in expressions or assignments.

Types of Type Conversion

Java supports two types:

- **Implicit Type Conversion (Widening)**
- **Explicit Type Casting (Narrowing)**

Implicit Type Conversion (Widening)

Definition

Implicit conversion is done automatically by the compiler when a smaller data type is converted into a larger data type.

Hierarchy (Widening Order)

byte → short → int → long → float → double

Syntax

larger_type variable = smaller_type_value;

Characteristics

- Done automatically by compiler
- No data loss
- Safe and reliable
- Happens between compatible types

Example (Implicit Conversion)

```
public class ImplicitExample {
    public static void main(String[] args) {
        int a = 10;
        double b = a;   // implicit conversion

        System.out.println("Value of a: " + a);
        System.out.println("Value of b: " + b);
    }
}
```

Output:

```
Value of a: 10
Value of b: 10.0
```

Explicit Type Casting (Narrowing)

Definition

Explicit type casting is manually performed by the programmer to convert a larger data type into a smaller data type.

Syntax

target_type variable = (target_type) value;

Characteristics

- Done manually by programmer
- May cause data loss
- Used when types are incompatible
- Less safe compared to implicit conversion

Example (Explicit Conversion)

```
public class ExplicitExample {
   public static void main(String[] args) {
      double a = 10.75;
      int b = (int) a;   // explicit casting

      System.out.println("Value of a: " + a);
      System.out.println("Value of b: " + b);
   }
}
```

Output:

```
Value of a: 10.75
Value of b: 10
```

Difference Between Implicit and Explicit Conversion

Implicit Conversion (Widening)	**Explicit Conversion (Narrowing)**
Performed automatically by compiler	Done manually by programmer
Converts smaller data type to larger data type	Converts larger data type to smaller data type
No data loss occurs	Data loss may occur
Safe and reliable	Risky if not handled properly
No special syntax required	Requires casting using (type)
Example: int → double	Example: double → int

Additional Points

- Implicit conversion occurs only between **compatible data types**
- Explicit casting is required for **larger to smaller conversions**
- Boolean cannot be converted to any other type
- Precision may be lost in narrowing conversion

- Type casting is widely used in expressions and calculations

Conclusion

Type conversion in Java allows smooth handling of different data types. Implicit conversion provides safe automatic conversion, while explicit casting gives flexibility but must be used carefully to avoid data loss.

Q. Discuss the selection (decision-making) statements in Java with suitable examples.

Answer

Definition

Selection statements in Java are used to control the flow of execution based on conditions. They allow the program to choose different paths depending on whether a condition is true or false.

Types of Selection Statements in Java

- if statement
- if-else statement
- else-if ladder
- switch statement

---

1. if Statement

Syntax

```
if (condition) {
   // statements
}
```

Explanation

Executes the block only if the condition is true.

Example

```
public class IfExample {
   public static void main(String[] args) {
      int a = 10;
      if (a > 5) {
         System.out.println("a is greater than 5");
      }
```

```
    }
}
```

Output

a is greater than 5

---

2. if-else Statement

Syntax

```
if (condition) {
    // true block
} else {
    // false block
}
```

Explanation

Executes one block if condition is true, otherwise executes another block.

Example

```
public class IfElseExample {
    public static void main(String[] args) {
        int num = 4;
        if (num % 2 == 0) {
            System.out.println("Even");
        } else {
            System.out.println("Odd");
        }
    }
}
```

Output

Even

---

3. else-if Ladder

Syntax

```
if (condition1) {
    // block1
} else if (condition2) {
    // block2
} else {
    // default block
}
```

Explanation

Used when multiple conditions need to be checked. Only one block is executed.

Example

```
public class ElseIfExample {
    public static void main(String[] args) {
        int marks = 75;

        if (marks >= 90) {
            System.out.println("Grade A");
        } else if (marks >= 60) {
            System.out.println("Grade B");
        } else {
            System.out.println("Grade C");
        }
    }
}
```

Output

Grade B

---

4. switch Statement

Syntax

```
switch (expression) {
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    default:
        // statements
}
```

Explanation

Used to select one block from multiple options based on the value of an expression.

Example

```
public class SwitchExample {
    public static void main(String[] args) {
        int day = 2;

        switch (day) {
            case 1:
```

```
            System.out.println("Monday");
            break;
        case 2:
            System.out.println("Tuesday");
            break;
        default:
            System.out.println("Other Day");
    }
  }
}
```

Output

Tuesday

Additional Points

- if is used for simple conditions
- if-else handles two choices
- else-if ladder is used for multiple conditions
- switch is preferred when comparing one variable with multiple values
- break is important in switch to prevent fall-through

Conclusion

Selection statements in Java help in decision-making and control the execution flow of a program. They make programs flexible and capable of handling different conditions efficiently.

Q. Describe the looping statements in Java with suitable examples.

Definition

Looping statements in Java are used to execute a block of code repeatedly as long as a specified condition is true. They help reduce code repetition and improve efficiency.

Types of Looping Statements in Java

- for loop
- while loop
- do-while loop

1. for Loop

Syntax

```
for (initialization; condition; update) {
   // statements
}
```

Explanation

- Initialization → executed once
- Condition → checked before each iteration
- Update → executed after each iteration

Example

```
public class ForExample {
   public static void main(String[] args) {
      for (int i = 1; i <= 5; i++) {
         System.out.println(i);
      }
   }
}
```

Output

```
1
2
3
4
5
```

---

2. while Loop

Syntax

```
while (condition) {
   // statements
}
```

Explanation

- Condition is checked before execution
- Loop runs only if condition is true

Example

```
public class WhileExample {
   public static void main(String[] args) {
      int i = 1;
```

```
        while (i <= 5) {
            System.out.println(i);
            i++;
        }
    }
}
```

Output

```
1
2
3
4
5
```

---

3. do-while Loop

Syntax

```
do {
    // statements
} while (condition);
```

Explanation

- Executes at least once
- Condition is checked after execution

Example

```
public class DoWhileExample {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println(i);
            i++;
        } while (i <= 5);
    }
}
```

Output

```
1
2
3
4
5
```

Difference Between Loops

for Loop	while Loop	do-while Loop
Entry-controlled	Entry-controlled	Exit-controlled
Used when iterations are known	Used when condition-based	Executes at least once
Initialization inside loop	Initialization outside loop	Initialization outside loop

Additional Points

- Loops reduce code repetition
- Infinite loops occur when condition never becomes false
- Nested loops are allowed
- break → exits loop
- continue → skips current iteration

Conclusion

Looping statements in Java allow repeated execution of code based on conditions. They are essential for efficient programming and are widely used in real-world applications.

Q. Write a Java program to remove duplicate elements from an integer array.

Answer

Definition

Removing duplicates from an array means eliminating repeated elements so that each element appears only once.

Approach

- Traverse the array
- Compare elements
- Store only unique elements
- Print the result

Program (Using Simple Nested Loops)

```
public class RemoveDuplicates {
    public static void main(String[] args) {
        int[] arr = {1, 2, 2, 3, 4, 4, 5};
        int n = arr.length;

        System.out.print("Array after removing duplicates: ");

        for (int i = 0; i < n; i++) {
            boolean isDuplicate = false;

            for (int j = 0; j < i; j++) {
                if (arr[i] == arr[j]) {
                    isDuplicate = true;
                    break;
                }
            }

            if (!isDuplicate) {
                System.out.print(arr[i] + " ");
            }
        }
    }
}
```

Output

Array after removing duplicates: 1 2 3 4 5

Alternative Method (Using Set – Advanced)

```
import java.util.*;

public class RemoveDuplicatesSet {
    public static void main(String[] args) {
        int[] arr = {1, 2, 2, 3, 4, 4, 5};

        Set<Integer> set = new LinkedHashSet<>();

        for (int num : arr) {
            set.add(num);
        }

        System.out.println("After removing duplicates: " + set);
    }
}
```

Output After removing duplicates: [1, 2, 3, 4, 5]

Additional Points

- Nested loop method → no extra memory, but slower
- Set method → faster, but uses extra space
- LinkedHashSet maintains insertion order
- Time Complexity:
  - Nested loops → $O(n^2)$
  - Set method → $O(n)$

Conclusion

Duplicate elements in an array can be removed using different approaches. The nested loop method is simple, while the Set approach is more efficient and commonly used in real-world applications.

Q. Briefly explain different Data Types in Java along with memory size.

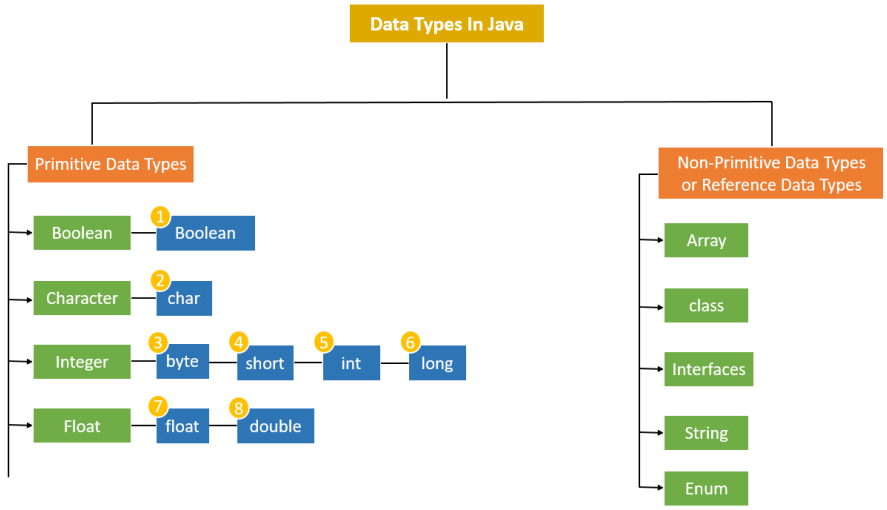
Definition

Java data types define the type of data a variable can store and the amount of memory it occupies.

Classification of Data Types

Java data types are classified into:

- **Primitive Data Types**
- **Non-Primitive (Reference) Data Types**



---

1. Primitive Data Types

Primitive data types store **actual values** and have fixed memory size.

- **byte**
  Stores whole numbers from –128 to 127
  Memory: 1 byte
- **short**
  Stores whole numbers from –32,768 to 32,767
  Memory: 2 bytes
- **int**
  Stores whole numbers (most commonly used)
  Memory: 4 bytes
- **long**
  Stores very large whole numbers
  Memory: 8 bytes
- **float**
  Stores decimal values (single precision)
  Memory: 4 bytes
- **double**
  Stores decimal values (double precision)
  Memory: 8 bytes
- **char**
  Stores a single character (Unicode)
  Memory: 2 bytes
- **boolean**
  Stores true or false
  Memory: ~1 byte (JVM dependent)

Example (Primitive Data Types)

```
public class PrimitiveExample {
   public static void main(String[] args) {

      int a = 10;
      double b = 20.5;
      char c = 'A';
      boolean d = true;

      System.out.println(a);
      System.out.println(b);
      System.out.println(c);
      System.out.println(d);
```

```
    }
}
```

Output

```
10
20.5
A
true
```

---

2. Non-Primitive (Reference) Data Types

Non-primitive data types store **reference (address)** of objects and have no fixed memory size.

- **String**
  Stores a sequence of characters
  Memory: Depends on number of characters
- **Array**
  Stores multiple values of the same type
  Memory: Depends on size and data type
- **Class / Interface**
  Used to create objects and define behavior
  Memory: Depends on object size

Example (Non-Primitive Data Types)

```
public class ReferenceExample {
    public static void main(String[] args) {

        String name = "Java";
        int[] arr = {1, 2, 3};

        System.out.println(name);
        System.out.println(arr[0]);
    }
}
```

Output

```
Java
1
```

Conclusion

Primitive data types have fixed memory sizes and are used for basic values, whereas non-primitive data types use variable memory depending on object creation and are used for complex data structures.

Q. Explain arrays in Java. Discuss one-dimensional and multi-dimensional arrays with examples.

Answer

Definition

An array in Java is a collection of elements of the **same data type** stored in **contiguous memory locations**. It is used to store multiple values in a single variable.

Declaration and Creation of Array

Syntax

```
dataType[] arrayName;          // declaration
arrayName = new dataType[size]; // creation
```

Example

```
int[] arr = new int[5];
```

---

1. One-Dimensional Array

Definition

A one-dimensional array stores elements in a **single row (linear form)**.

Initialization

```
int[] arr = {10, 20, 30, 40, 50};
```

Example Program

```
public class OneDArray {
   public static void main(String[] args) {

      int[] arr = {10, 20, 30, 40, 50};

      for (int i = 0; i < arr.length; i++) {
         System.out.println(arr[i]);
      }
   }
}
```

Output

```
10
20
30
40
50
```

---

2. Multi-Dimensional Array

Definition

A multi-dimensional array is an array of arrays. The most common type is a **two-dimensional array (matrix)**.

Syntax

```
dataType[][] arrayName = new dataType[rows][columns];
```

Initialization

```
int[][] arr = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Example Program

```
public class TwoDArray {
    public static void main(String[] args) {

        int[][] arr = {
            {1, 2, 3},
            {4, 5, 6}
        };

        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                System.out.print(arr[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output

```
1 2 3
4 5 6
```

Key Points

- Arrays store elements of the **same data type**
- Index starts from **0**
- Size of array is **fixed**
- Stored in **contiguous memory locations**
- Accessed using index

Difference

One-Dimensional Array	**Multi-Dimensional Array**
Stores elements in single row	Stores elements in rows and columns
Uses one index	Uses multiple indices
Example: arr[i]	Example: arr[i][j]

Conclusion

Arrays in Java provide an efficient way to store and access multiple values of the same type. One-dimensional arrays are simple linear structures, while multi-dimensional arrays are used for complex data representation like matrices.